

GamutX

Python Modules

PYTHON SCRIPTING

Introduction to Modules

- Revision of Function
- What is Module ?
- Access Modules Functions & Variables

Revision of Function

Q - What is Function & How to Create a Function ?

- A function is a block of **reusable code** that performs a **specific task**.
- Functions can accept **inputs (parameters)** and return **outputs (results)**.

Creating a Function

Let's create a simple function called somefun:

```
def somefun():  
    print("something")
```

Function Name

use to create Function

Q - How to Call a Function ?

We can call the function and try to **store** its **output** in a **variable**. Here's how we do that:

```
def somefun():  
    print("something")
```

Creating Function

```
x = somefun()  
print(x)
```

Calling Function

Storing Function

Output

```
something  
None
```

Since the function doesn't **return** anything, **x** is automatically set to **None**

Q - Checking the Type of x ?

Let's see how to check type of x

```
def somefun():  
    print("something")
```

```
x = somefun()  
print(x)
```

```
print(type(x))
```

Use to Check Type of Object

```
# something  
# None  
# <class 'NoneType'>
```

- Since the function doesn't **return** anything, **x** is **automatically** set to **None**
- **None** is a special value in **Python** that represents "**nothing**"
- The type of None is **NoneType**

NOTE:

- In Python, everything is an **object**. Objects are like containers that hold data and functionality. They can have variables, functions, and even classes

What is Module ?

A **module** is just a **Python file (.py)** that contains code — like:

- **Functions**
- **Variables**
- **Classes**

We can **import** that file into other **scripts** to **reuse** the code it **contains**.

Why Use Modules?

- **Organize** your code.
- **Avoid repeating** the same code in multiple files.
- **Reuse** logic across multiple scripts.

Creating a module

Here, we create a Python file called **mymodule.py** with some basic functions:

Folder Structure

```
project_folder/  
├── mymodule.py  
└── script.py
```



Creating a Module (mymodule.py)

```
def sonefun():  
    print("something")  
  
def sonefun1():  
    print("something")  
  
def sonefun2():  
    print("something")
```

- We Define 3 Functions

We import and use function from **mymodule.py**

Using Module

Importing the Module in Another File (script.py)

- Now, in another file, say **script.py**,
- you can **import** the mymodulemodule to use the functions defined inside **mymodule.py**:

script.py,

```
import mymodule
```

Custom Module

```
print(mymodule)
```

```
mymodule.sonefun()
```

Calling function from mymodule

Output

Module Name

Path to module

```
<module 'mymodule' from 'path/to/mymodule.py'>  
something
```

Access Modules Functions & Variables

- When you import a module in Python:
- You get access to everything defined inside it — like:
 -  **Functions**
 -  **Variables**
 -  **Classes**
- To use them, **prefix** with the module name, followed by a **dot** .

Basic Syntax:

```
import module_name
```

Custom Module

```
module_name.function_name()
```

Call a function from the module

```
module_name.variable_name
```

Access a variable from the module

module_1.py

```
some_var = 10
def funtion1():
    print("funtion1")
def funtion2():
    print("funtion2")
def funtion3():
    print("funtion3")
def funtion4():
    print("funtion4")
```

script.py

```
import module_1
module_1.funtion1()
print(module_1.some_var)

# funtion1
# 10
```

Importing Module

Calling Function

- **Variable:** `some_var` initialized to 10.
- **Functions:**
 - `funtion1()`, `funtion2()`, `funtion3()`, and `funtion4()`,
- each printing its name.

Key Points

- **Importing the Module:**

- Always start by importing the module using `import module_name`.
- This allows you to access all the **functions** and **variables** defined in that module.

- **Accessing Functions:**

- Use the syntax `module_name.function_name()` to call functions from the module.
- This helps **avoid naming conflicts** and **clarifies the origin of the function**.

- **Accessing Variables:**

- Similarly, use `module_name.variable_name` to access variables defined in the module.
- This keeps the **code clean and understandable**.

Where to Import Module From

- Importing a Module from the Same Folder
- Importing a Module from a Subfolder
- Importing Multiple Modules

Importing a Module from the Same Folder

- This is the most **basic** and **common case** when working on a Python project:
- Our **module** and **script** are in the same folder.

Folder Structure



- When we write `import test_module`,
- Python looks for a **file named test_module.py in the same folder as main.py**.

Importing a Module from a Subfolder

Now let's look at how to import a module when it's **located inside** a **subfolder**.

Folder Structure

```
project/  
├── demo/  
│   ├── config.py  
│   └── main.py
```

The main script

The module (inside a subfolder)

config.py

```
def show_message():  
    print("Hello from config.py!")
```

main.py

```
from demo import config  
  
config.show_message()  
  
# Hello from config.py !
```

- A folder named **demo/**
- Inside it, a file named **config.py**

Importing Multiple Modules

- As your project **grows**, you'll want to organize related code into **separate folders** and files (**modules**).
- Now let's look at how to import and use functions from multiple modules **stored** in **subfolders**.

Folder Structure

```
project/  
├── utilities/  
│   ├── math/  
│   │   └── math_utils.py  
│   ├── string/  
│   │   └── string_utils.py  
└── main.py
```

main script

math_utils.py

```
def add(a, b):  
    return a + b  
def subtract(a, b):  
    return a - b
```

string_utils.py

```
def to_uppercase(s):  
    return s.upper()  
def to_lowercase(s):  
    return s.lower()
```

Folder Structure

```
project/  
├── utilities/  
│   ├── math/  
│   │   └── math_utils.py  
│   ├── string/  
│   │   └── string_utils.py  
└── main.py
```

main script

Folder

Subfolder

Module

```
from utilities.math import math_utils  
from utilities.string import string_utils
```

```
result_add = math_utils.add(10, 5)  
result_subtract = math_utils.subtract(10, 3)
```

Math Module Functions

```
uppercase = string_utils.to_uppercase("hello")
```

Convert to uppercase

```
lowercase = string_utils.to_lowercase("WORLD")
```

Convert to lowercase

```
print(f"Addition: {result_add}")    # Addition: 15  
print(f"Subtraction: {result_subtract}") # Subtraction: 7  
print(f"Uppercase: {uppercase}")    # Uppercase: HELLO  
print(f"Lowercase: {lowercase}")    # Lowercase: world
```

Key Points

- Module Import Statements Also known as: **Module Import Syntax**

1. import module

```
import module_name
```

- Use when your **script** and **module** are in the **same directory**.

2. import modules from Subfolder

```
from folder_name import module_name
```

- Use when your module is inside a **subfolder** of your project.

3. Importing Multiple Modules

```
from utilities.math import math_utils
```

- Use when you **want to import multiple modules**

Common Ways of Importing Modules

- Importing the Entire Module as Is
- Importing Specific Functions from a Module
- Importing Multiple Functions from a Module
- Importing All Functions from a Module
- Importing a Module with an Alias
- Importing Modules Inside Folders with an Alias

Common Ways of Importing Modules

Ways to write import statements (Module Import Statements)

- Importing the Entire Module as is
- Importing Specific Functions from module
- Importing Multiple Functions from module
- Importing All Functions from module
- Importing module with an Alias
- Importing Modules Inside Folders with an Alias

Project Setup

- This setup demonstrates **common ways** of importing **modules** in **Python** —
 - from the **same folder**, from **subfolders**, and across a **structured project**.

Project Structure

```
repo/  
├── func/  
│   └── module2.py  
├── module1.py  
└── script.py
```

module1.py

```
module_variable = "This is a variable from module3 in the project folder."  
def root_function():  
    print("This is a function from module1 in the root folder.")  
def greet():  
    print("Hello Everyone !")
```

- Defines a **variable** and **two functions**
- Located directly in the project root

module2.py (in the func folder)

```
def func_function1():  
    print("This is function 1 from module4 in the func folder.")  
def func_function2():  
    print("This is function 2 from module4 in the func folder.")  
func_variable = "This is a variable from module4 in the func folder."
```

- Defines **two** functions and **one** variable
- Located inside the **func** subfolder

Importing the Entire Module as Is

- This is the most **straightforward** way to bring a module into **your script**.
- You import the whole **module**, and access its contents using the module name as a **prefix**.

Syntax

```
import module_name
```

Module

Example: **script.py**

```
import module1 -----> Loads the entire module1.py file into script
```

```
module1.root_function() -----> Calls the Function from the module1
```

```
# Output  
# This is a function from module1 in the root folder.
```

✓ Why Use This Method?

- ✓ Simple to write and understand
- ✓ Keeps the code organized by making it clear where each function comes from
- ✓ Useful for small or self-contained modules

Importing Specific Functions from a Module

- If we only need a specific function from a module, we can import just that function.

Syntax

Module

```
from module_name import function_name
```

Imports only the Specific function

Example: script.py

```
from module1 import greet
```

import only specific functions

```
greet() -----> Calls the function directly
```

```
# Hello Everyone !
```

✓ Why Use This Method?

- ✓ **Cleaner:** Only brings in what you need
- ✓ **Simpler** calls: No need to prefix with the module name
- ✓ **Focused:** Makes the script easier to read and maintain

Importing Multiple Functions from a Module

- Sometimes, we need a **few** specific **functions** but don't want to import the **entire module**.
- We can import them by **name**, separated by **commas**.

Syntax

Module

```
from module_name import function_name1, function_name2
```

Importing two specific functions

Example: script.py

```
from module1 import root_function, greet
```

```
root_function() ←
```

```
greet() ←
```

```
# This is a function from module1 in the root folder.  
# Hello Everyone !
```

Call both Function directly

✓ Why Use This Method?

- Only imports what you need
- Keeps your code clean and easy to read
- No need to write the module name every time
- Good for small, focused scripts

Importing All Functions from a Module (*** Import**)

- You can import everything from a module at once **using ***.
- This gives you access to all **functions** and **variables** without using the **module name**.

Syntax

```
from module_name import *
```

Import everything

Example: script.py

```
from module1 import *
```

```
greet() ----->
```

Call greet directly

```
print(module_variable) -->
```

Access variable directly

```
# Hello Everyone !
```

```
# This is a variable from module1 in the project folder.
```

! Why Be Careful?

- This method can lead to **confusion**,
- especially if multiple modules have functions or variables with the **same name**.
- It's best to **avoid** this **in larger projects**.

Importing a Module with an Alias

- If the module name is long or used many times, you can assign it a shorter name (alias) to make your code cleaner.

Syntax

```
import module_name as alias
```

Custom name given to module

Example: script.py

```
import module1 as m4
```

alias for module1

```
m4.root_function()  
m4.greet()
```

Calling using alias

```
# This is a function from module1 in the root folder.  
# Hello Everyone !
```

Why Use an Alias?

- ✓ Makes **long** names **short** and **easy** to type
- ✓ Keeps code **cleaner** and **easier** to read
- ✓ Helpful when the module is used **frequently**

When to Use

- ✓ With modules that have long names
e.g., import **matplotlib.pyplot** as **plt**
- ✓ In larger scripts where you use the same module many times

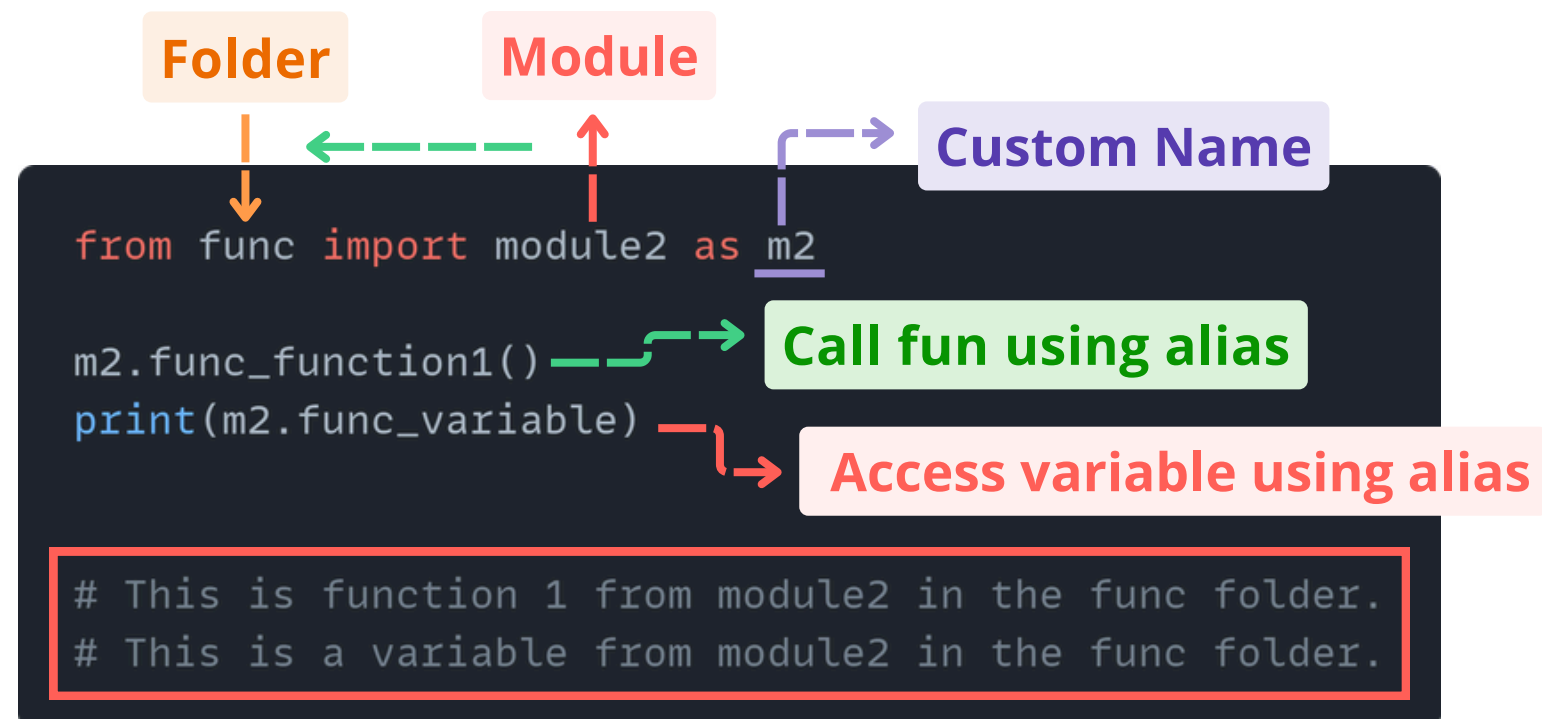
Importing Modules Inside Folders(Sub Folders) with an Alias

- When a module is **inside** a **folder**, you can **import** it using the **folder** name.
- Using an alias makes your code **shorter** and **easier** to **manage**.

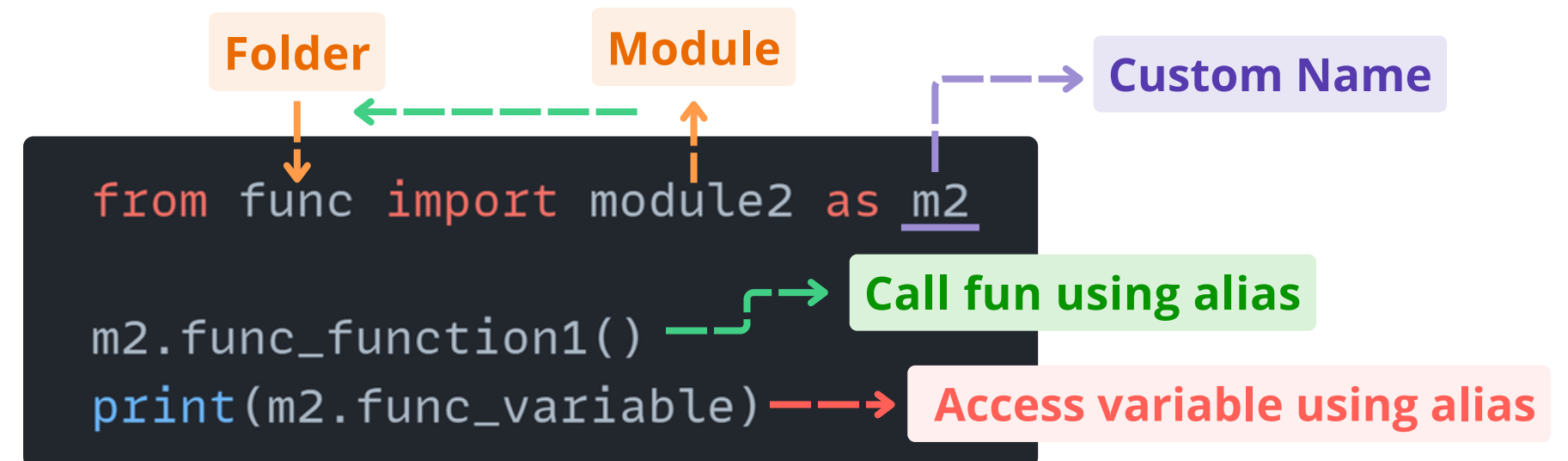
Syntax

```
from folder_name import module_name as alias
```

Import module2 from 'func' folder with alias 'm2'



Example: script.py



✓ Why Use This Method?

- ✓ Keeps your code organized in folder-based projects
- ✓ Shorter names make repeated calls quicker and cleaner
- ✓ Easy to understand where things are coming from



H0. Create a Custom Module & Import It



Objective

- Learn how to create your own Python module (**.py file**) and import it into another script to reuse functions and keep your code organized.



Instructions:

To complete this task:

- Create a Python module file (**my_module.py**) that contains a few simple functions.
- Create a second Python script (**main_script.py**) to **import and use** that module.
- **Run the script** to test if the functions work as expected.

 Steps to Complete the Task**Step 1: Create a Module**

- Open **VS Code** (or your preferred editor).
- Create a new file named **my_module.py**.
- Add the following code:

```
def greet(name):  
    return f"Hello, {name}!"  
  
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

- Save this file.

Step 2: Create and Use the Module:

- In the **same folder**, create another Python file named **main_script.py**.
- Add the following code:

```
import my_module

print(my_module.greet("Alice"))
print(f"Addition: {my_module.add(10, 5)}")
print(f"Subtraction: {my_module.subtract(10, 5)}")
```

- Save the file and run **main_script.py**.

Step 3: Expected Output

```
Hello, Alice!
Addition: 15
Subtraction: 5
```

What to Do:

- Share your screen and run the code live during the session.
- Show how importing the module works and explain each function.



H0. Create an Addition & Subtraction Module

Objective

- Develop a standalone module focusing on addition and subtraction operations.

Instructions:

To complete this task:

- Create a module named **math_operations.py** containing **add** and **subtract** functions.
- Create another script **test_math_operations.py** to **import and test** the module.
- Run the script and **verify the output**.
- (Optional Challenge) Extend the module by adding **multiplication** and **division** functions.

 Steps to Complete the Task**Step 1: Create a Module**

- Open **VS Code** or any editor.
- Create a file named **math_operations.py**.
- Add this code:

```
def add(a, b):  
    return a + b  
  
def subtract(a, b):  
    return a - b
```

- Save the file.

Step 2: Create the Test Script:

- In the **same folder**, create a new file: test_math_operations.py.
- Add the following code:

```
import math_operations

result_add = math_operations.add(7, 3)
print(f"7 + 3 = {result_add}")

result_subtract = math_operations.subtract(7, 3)
print(f"7 - 3 = {result_subtract}")
```

- Save and run this script.

Step 3: Expected Output:

```
7 + 3 = 10
7 - 3 = 4
```

Step 4: Bonus Challenge:

- add two more functions to `math_operations.py`:
 - `multiply(a, b)`
 - `divide(a, b)`
- Update `test_math_operations.py` to test these new functions.

Step 5: What to Do:

- Run the code during the live session and show the output.
- 



H0. Using Custom Addition & Subtraction Module

Objective

Learn how to create and use a custom Python module by practicing different ways to import and call its functions, including :

- Create a Custom Module
- Importing the Entire Module
- Importing Specific Functions from the Module:
- Importing All Functions from the Module (Use Sparingly):
- Using Aliases for Modules and Functions:

Instructions:

To complete this task:

- Create a custom module file (**my_module.py**) with three simple functions.
- In separate examples, use **various import styles** to access and use those functions.
- Observe how function access changes depending on the import method.
- Discuss why some import styles are better than others.

🎯 Steps to Complete the Task

Writing a Module (Creating a .py File)

- A module in Python is simply a file containing Python definitions and statements. Here's how you can create a custom module:

Create a .py file:

- Open your **text editor** or IDE.
- Write your Python code in this file.
- Save the file with a **.py extension**.

For example, let's create a module called **my_module.py**:

```
def greet(name):  
    return f"Hello, {name}!"  
def add(a, b):  
    return a + b  
def subtract(a, b):  
    return a - b
```

- In this example, **my_module.py** contains **three** functions: **greet**, **add**, and **subtract**.
- Once you have created your module, you can **import** and use it in **another Python script**. Here's how:

Importing the Entire Module:

- Use the **import** statement to import the module.

Access the functions using the module name.

```
import my_module
print(my_module.greet("Alice"))
print(my_module.add(5, 3))
print(my_module.subtract(5, 3))
```

Output: Hello, Alice!
Output: 8
Output: 2

- In this script, **my_module** is imported, and its functions are accessed using the dot notation (**my_module.function_name**).

Importing Specific Functions from the Module:

- Use the `from module_name import function_name` syntax to import specific functions.

Example:

```
from my_module import greet, add
print(greet("Bob"))      # Output: Hello, Bob!
print(add(10, 7))         # Output: 17
```

- Here, only the `greet` and `add` functions are imported from `my_module`, and they can be used directly without the module name prefix.

Importing All Functions from the Module (Use Sparingly):

- Use the from **module_name** import * **syntax** to import all functions.

Example:

```
from my_module import *  
print(greet("Charlie"))  
print(add(15, 5))  
print(subtract(15, 5))
```

```
# Output: Hello, Charlie!  
# Output: 20  
# Output: 10
```

- This imports all functions from **my_module**, but it's generally not recommended because it can lead to **namespace conflicts** and make the code **less readable**.

Using Aliases for Modules and Functions:

- Use the **as** keyword to create an alias for a module or function.

Example:

```
import my_module as mm  
print(mm.greet("Diana"))  
print(mm.add(20, 10))
```

```
# Output: Hello, Diana!  
# Output: 30
```

- Here, **my_module** is imported with an alias **mm**, making it **easier** to **reference**.

Best Practices for Importing Modules

- Use `import module_name` for clarity
- Use `from module_name import function_name`
- Avoid `from module_name import *`

Use `import module_name` for clarity

Why?

- This approach keeps the code **clear** and **avoids** potential **naming conflicts**.
- It explicitly **states** where each **function** or **variable** comes from.
- It's useful in **large projects** where **multiple modules** may have functions with the **same name**.

Example:

```
import math -----> Importing the entire math module

radius = 5
area = math.pi * (radius ** 2) -----> Here we use pi from math module
print(area)

# 78.53981633974483
```

- **math.pi** makes it obvious that **pi** comes from the **math module**
- If **another module** also had a **pi variable**, this approach **avoids confusion**

Avoid from module_name import *

Why?

- ✗ **Namespace pollution:** You **can't tell** which **function** or **variable** came **from where**
- ✗ **Higher chance of conflicts:** Especially when **different modules** have the **same function names**
- ✗ **Harder to debug:** You **lose visibility** into which **module** a **function** or variable **belongs** to

Example (**Bad Practice**):

```
from math import *
```

→ import everything

```
print(pi)
```

→ Works, but unclear where pi came from

```
print(sqrt(25))
```

→ Works, but might conflict with other sqrt functions

```
# 3.141592653589793  
# 5.0
```

- If another module has a **sqrt()** function,
- it can overwrite **math.sqrt** without warning.

✓ When It's Acceptable

- In small scripts or quick testing
- In interactive environments (like **Jupyter Notebooks** or **Python shell**)

Use from module_name import function_name

Why?

- Makes **code shorter** and **cleaner**
- **No** need to write the **module name** each time
- Best for **importing** a few specific **functions** or **variables**

Example:

The diagram illustrates the use of the `from module_name import function_name` syntax. It shows a code block with the following content:

```
from math import pi, sqrt

radius = 5
area = pi * (radius ** 2)
print(area)

print(sqrt(16))
```

Annotations:

- Module:** A red dashed arrow points from the `math` module in the import statement to the `Module` label.
- Variable:** An orange dashed arrow points from the `pi` variable in the import statement to the `Variable` label.
- Function:** A green dashed arrow points from the `sqrt` function in the import statement to the `Function` label.

Output:

```
# 78.53981633974483
# 4.0
```

- This keeps **code explicit**. we know where **pi** and **sqrt** come from
- It avoids **repeating math**. every time we call a **function**

Summary

Import Style

- **import module_name**
- **from module_name import function_name**
- **from module_name import ***

Pros

- Avoids conflicts, clear
- Shorter, cleaner code
- Convenient in quick tests

Cons

- Longer syntax (module_name.function())
- Can lead to conflicts if overused
- Can overwrite existing functions, makes debugging harder

Best Practice Recommendation:

- ✓ Use import module_name for most cases.
- ✓ Use from module_name import function_name when you only need a few specific items.
- ✗ Avoid from module_name import * in large projects.

Thank You

